

Data Structures

CSE-207

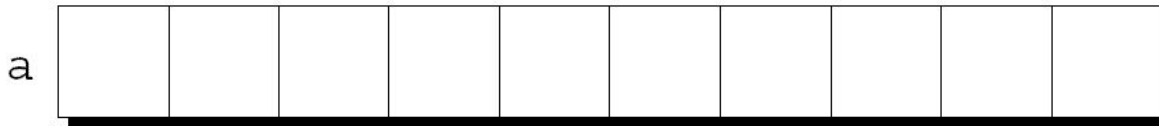
Md. Manowarul Islam
Dept. of CSE, Jagannath University

Programming Consideration....

- Array

One-Dimensional Arrays

- An **array** is a data structure containing a number of data values, all of which have the same type.
- The simplest kind of array has just one dimension.
- The elements of a one-dimensional array a are conceptually arranged one after another in a single row (or column):



Defining Arrays

■ When defining arrays, specify

- Name
- Type of array (Array can be defined over any type)
- Number of elements

```
arrayType arrayName[ numberOfElements ];
```

- Examples:

```
float grade[ 7 ];
```

```
int c[ 10 ];
```

■ Defining multiple arrays of same type

- Format similar to regular variables
- Example:

```
int b[ 100 ], x[ 27 ];
```

Array Elements

Name of array (Note that all elements of this array have the same name, c)

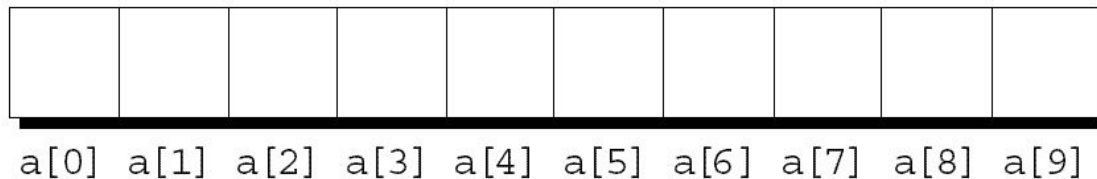
- To refer to an element, specify
 - *Array name* for the collection
 - *Position number* for the member
- Format:
 - arrayname* [*position number*]
 - First element at position 0
 - n element array named c:
 - c[0], c[1]...c[n - 1]

c[0]	-45
c[1]	6
c[2]	0
c[3]	72
c[4]	1543
c[5]	-89
c[6]	0
c[7]	62
c[8]	-3
c[9]	1
c[10]	6453
c[11]	78

Position number of the element within array c

Array Subscripting

- To access an array element, write the array name followed by an integer value in square brackets.
- This is referred to as *subscripting* or *indexing* the array.
- The elements of an array of length n are indexed from 0 to $n - 1$.
- If a is an array of length 10, its elements are designated by $a[0]$, $a[1]$, ..., $a[9]$:



Array Subscripting

- Expressions of the form `a[i]` can be used in the same way as ordinary variables:

```
a[0] = 1;  
printf("%d\n", a[5]);  
++a[i];
```

- In general, if an array contains elements of type T , then each element of the array is treated as if it were a variable of type T .

Array Subscripting

- Many programs contain `for` loops whose job is to perform some operation on every element in an array.
- Examples of typical operations on an array `a` of length `N`:

```
for (i = 0; i < N; i++)  
    a[i] = 0;                /* clears a */
```

```
for (i = 0; i < N; i++)  
    scanf("%d", &a[i]);      /* reads data into a */
```

```
for (i = 0; i < N; i++)  
    sum += a[i];             /* sums the elements of a */
```


Array

- Find sum
- Find average
- Find max
- Find min
- Print in reverse order

Algorithm Analysis

- Data structures
 - Methods of organizing data in order to facilitate access and modification.
- What is Algorithm?
 - a clearly specified **set of simple instructions on the data** to be followed to solve a problem
 - Takes a set of values, as input and
 - produces a value, or set of values, as output
 - May be specified
 - In English
 - As a computer program
 - As a pseudo-code
- Program = data structures + algorithms

Algorithm Analysis

- Why need algorithm analysis ?
 - ❑ writing a working program is not good enough
 - ❑ The program may be inefficient!
 - ❑ If the program is run on a **large data set**, then the running time becomes an issue

Algorithm Analysis

- We only analyze **correct** algorithms
- An algorithm is correct
 - If, for every input instance, it halts with the correct output
- Incorrect algorithms
 - Might not halt at all on some input instances
 - Might halt with other than the desired answer

Complexity..

- Predict the amount of resources required:
 - **memory**: how much space is needed?
 - **computational time**: how fast the algorithm runs?
- **FACT**: running time grows with the size of the input
- Input size (number of elements in the input)
 - Size of an array, polynomial degree, # of elements in a matrix, # of bits in the binary representation of the input, vertices and edges in a graph

Def: Running time = the number of primitive operations (steps) executed before termination

- Arithmetic operations (+, -, *), data movement, control, decision making (*if, while*), comparison

Complexity..

- We define complexity as a function $f(n)$
- **Worst Case Complexity:**
 - the function defined by the *maximum* number of steps taken on any instance of size n
- **Best Case Complexity:**
 - the function defined by the *minimum* number of steps taken on any instance of size n
- **Average Case Complexity:**
 - the function defined by the *average* number of steps taken on any instance of size n

Searching Algorithm

- Depends on the way the information in DATA is organized.
- Linear Search: when the data is unsorted or sorted
- Binary Search: when the data is **sorted only!!!**

10
25
35
68
78
15

Linear Search

10
15
25
35
68
75

Binary Search

Linear Search

- We have an array DATA with N elements.
- We have to find the LOC of ITEM (Say ITEM=15)

Working Process:

10
25
35
68
78
15

Check element one by one.

DATA[1]=ITEM ???

IF YES then OK.

Otherwise DATA[2]=ITEM ???

.....So on.

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

1st: $K=1$ $DATA[1]=10$.
 $15 \neq 10$ and $LOC=0$ so $K=2$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

2nd: $K=2$ $DATA[2]=25$.

$15 \neq 25$ and $LOC=0$ so $K=3$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

3rd: $K=3$ $DATA[3]=35$.

$15 \neq 35$ and $LOC=0$ so $K=4$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

4th: $K=4$ $DATA[4]=68$.
 $15 \neq 68$ and $LOC=0$ so $K=5$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

4th: $K=5$ $DATA[5]=78$.
 $15 \neq 78$ and $LOC=0$ so $K=6$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

4th: $K=6$ $DATA[6]=15$.
 $15==15$ and $LOC=6$

Linear Search

1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$ and $LOC=0$.
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
7. Exit.

10
25
35
68
78
15

4th: $K=6$ $DATA[6]=15$.
 $15==15$ and $LOC=6$

Complexity of linear Search

- Worst case:
 - Occurs when item is in last location of the array.
 - $C(n)=n$
- Average case:
 - We assume ITEM is in the array.
 - So it can be found at any location.
 - Number of comparison be any of $1,2,3,...n$.

Complexity of linear Search

- Average case:

- Each number occurs with probability $1/n$
- Number of comparison be any of $1, 2, 3, \dots, n$.

$$\begin{aligned}C(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + 3 \cdot \frac{1}{n} \dots + n \cdot \frac{1}{n} \\&= (1 + 2 + 3 \dots + n) \cdot \frac{1}{n} \\&= \frac{n(n+1)}{2} \cdot \frac{1}{n} \\&= \frac{n+1}{2}\end{aligned}$$

- Comparison is about half of total element.

END